Project Write-Up: Evaluating Iterative Password Hashing

Christian Giamalis

Abstract

This project investigates the security and performance implications of iterative password hashing using salted SHA-256. By varying the number of hash iterations from a single application to millions, the experiment measures median latency, attacker throughput, and slowdown factors relative to a baseline. Results demonstrate that increasing iterations proportionally reduces the number of guesses an attacker can perform per second, thereby strengthening resistance to brute-force attacks. At the same time, the experiment highlights the tradeoff between enhanced security and acceptable login latency for legitimate users. While iterative SHA-256 is not recommended for production systems, the study provides a reproducible framework for understanding how iteration counts influence attacker cost and serves as a foundation for comparing modern password-based key derivation functions such as PBKDF2, bcrypt, and Argon2.

1. Introduction

Password hashing is a cornerstone of secure authentication systems. The central question of this project is: does hashing a password multiple times enhance security compared to hashing once?

The motivation is practical: defenders want to slow down brute-force attackers without imposing unacceptable latency on legitimate users. Iterative hashing is one common technique, but its effectiveness and tradeoffs must be measured.

2. Threat Model

Attacker scenario: An offline attacker obtains a password database containing salts and stored digests. They attempt brute-force guesses by hashing candidate passwords with the same algorithm.

Defender goal: Increase per-guess cost so attacker throughput decreases, making brute force less feasible.

2a. Assumptions:

Attacker uses the same algorithm on comparable hardware.

No GPU/ASIC acceleration modeled (CPU-bound only).

Password entropy/distribution is outside scope; we focus on per-guess cost.

3. Methodology

Implementation

Language: Python 3

Hashing method: salted iterative SHA-256 (hashlib)

Iterations parameter: number of times the digest is re-hashed

Baseline: single salted SHA-256 (iterations = 1)

Harness: custom measurement utilities with warmup and repeated runs

Environment

Machine: 12th Gen Intel(R) Core i7-12700H, 16gb RAM @ 4800 MT/s, Windows 11, Python 3

Conditions: machine idle, VS Code terminal, venv activated

Procedure

1. Run experiments for iterations = [1, 10, 50, 100, 500, 1000, 10,000, 50,000, 1,000,000, 5,000,000].
2. Each experiment: warmup = 50, runs = 500.
    a. A warmup of 50 runs is done without recording data, as the first few runs often take longer than subsequent ones.
3. Collect median latency, mean, stdev.
4. Compute attacker throughput: guesses/sec = 1 / median_s.
5. Compute slowdown factor: median(iter) / median(1).
6. Repeat key configs 3× to check reproducibility (require rel stdev < 5%).

4. Results.

Iterations	Median Latency	Guesses/second	Slowdown vs x1 (base)
1	7.20e-06	1.39e+05	x1
10	1.03e-05	9.71e+04	x1.43
50	4.56e-05	2.19e+04	x6.33
100	9.06e-05	1.10e+04	x12.6
500	4.58e-04	2.18e+03	x63.6
1,000	1.01e-03	9.87e+02	x141
5,000	4.64e-03	2.15e+02	x645
10,000	9.80e-03	1.02e+02	x1,361
50,000	1.02e-01	9.76	x14,222
1,000,000	2.15	0.465	x298,000
5,000,000	4.69	0.213	x651,000

(results pasted from .json files)

5. Interpretation

Scaling behavior: Scaling is roughly linear: 10× iterations ≈ 10× slowdown, though variance grows at high counts.

Attacker cost: At 1 million iterations, each guess takes ~2.15 seconds, brute force throughput drops below 1 guess/sec.

Usability tradeoff: At 1 million iterations, each guess takes ~2.15 seconds, brute force throughput drops below 1 guess/sec.

Security implication: The security benefit of adding iterations comes directly from how much slower each guess becomes

6. Limitations

CPU-bound only: Does not model GPU/ASIC acceleration, where attackers gain more advantage.

No memory hardness: Iterative SHA-256 is CPU-bound; memory-hard KDFs (Argon2, scrypt) provide stronger resistance.

Single machine: Results specific to one environment; attacker hardware may differ.

Password strength: Iterations don't fix weak passwords; entropy remains critical.

7. Future Work

Add PBKDF2, bcrypt, scrypt, Argon2 to harness for broader comparison.

Test concurrency (parallel logins) to measure server cost.

Run on different hardware (multi-core, GPU) to compare attacker advantage.

Explore re-hash-on-use upgrade strategy (increase iterations over time).

8. Conclusion

Hashing a password multiple times does enhance security in the offline brute-force model by reducing attacker throughput. In our measurements, [insert iteration count] iterations slowed attackers by ~[insert slowdown factor]× with median latency of [insert latency].

.